

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

Distributed Sensor Monitoring with HTTP, MQTT, SNMP, and SQLite

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Embedded Systems

Contents

1 Scenario and system objective 2

2 General implementation and submission rules 2

3 Preliminary research: 10 percent 2

4 Required directory organization 2

5 Part 1: base distributed database, 30 percent 2

6 Part 2: two-layer storage and caching, 20 percent 3

7 Part 3: MQTT gateway, 15 percent 6

8 Part 4: SNMP sensor access, 15 percent 6

9 Part 5: dated sensor history API, 5 percent 6

10 Part 6: persistent alert daemon, 5 percent 6

11 Report and final-delivery checklist 7

1 Scenario and system objective

Act as the engineer of a hotel-monitoring system. Floors contain temperature, humidity, motion, and other environmental sensors. Each node receives and stores readings for its connected sensors. One master receives operator requests and returns the final result. Slave nodes communicate with the master and do not directly serve the operator. If the requested reading is absent locally, the request must continue to the next node until a result or a final not-found response returns through the master.

2 General implementation and submission rules

Use C or C++ only. Organize the six parts into separately numbered directories, each containing its own source, scripts, answers, guide, and report. Write the report from a system designer's perspective: architecture, components, communication, request/response paths, and decisions, rather than line-by-line code commentary. Include at least one architecture or data-flow diagram for each part. Every part requires a Markdown guide explaining dependencies, compilation, execution, requests, and tests. Use Makefiles. Supply a Bash build-and-run script; the original rules assign zero to a part lacking this script. Git is recommended and receives credit, although it is optional. Initialize databases from the supplied data files; evaluation files can change. Hard-coded answers or sensor values invalidate the exercise. IP addresses, ports, database locations, and execution settings must come from configuration, command-line arguments, or environment variables. AI assistance is permitted; evaluation focuses on system design, implementation correctness, executability, organization, and analysis. Provide a short final test video. Show each build/run script, requests and responses, the master/slave lookup, caching-time difference, MQTT exchange, SNMP readings, history API output, and alerts in the database. The requested video demonstrates operation and results; it should not explain internal code or architecture.

3 Preliminary research: 10 percent

Before implementing, explain Mongoose and how it supports a C/C++ HTTP server and API; distributed databases and several example systems; the meaning and use of an API; LAN communication among nodes; and distributed-system properties, benefits, and design challenges.

4 Required directory organization

Create parts 01 through 06. Parts 01, 02, and 03 each contain master, slave, and scripts directories, plus a report and README. Part 04 contains SNMP configuration and scripts. Part 05 contains the API and scripts. Part 06 contains the daemon, scripts, and service definitions. Internal organization may change when justified, but each part must remain independent and contain everything required to build, run, and test it.

5 Part 1: base distributed database, 30 percent

Create one master and two slaves as Ubuntu 22.04 virtual machines on the same network. Assign valid addresses and communicate by IP and port. Verify connectivity and include a network diagram. Give each node a local SQLite database and initialization script using the supplied files. Before implementation, inspect the schema and input data and explain them briefly. Implement master and slave programs. A request contains sensor type and identifier; the output is the latest recorded value. Mongoose may implement the HTTP API. Obtain addresses and ports dynamically. The operator

sends requests only to the master. The master searches locally, forwards a miss to slave 1, continues to slave 2 if necessary, and returns the result. If no node contains the reading, return an appropriate not-found message. Explain whether forwarding is performed by successive nodes or coordinated by the master; do not hide a difference between the requested and implemented path. The suggested master directory contains main.c/main.cpp, Makefile, master_init_db.sh, and config.example; the slave has corresponding source, Makefile, slave_init_db.sh, and example configuration. The scripts directory contains build_and_run.sh and test_requests.sh. The Markdown guide must explain dependency installation; compilation and execution of both node types; requests with curl; the database schema; topology; request/response path; HTTP security and improvements. Grading considers correct operation and responses, working automation, SQLite use, the diagram, dynamic addresses/ports, and security explanation. An optional advanced extension may manage nodes using one shared IP with different ports. Document its diagram, address/port settings, program execution, build/run script, and full Markdown instructions if implemented.

6 Part 2: two-layer storage and caching, 20 percent

Retain SQLite as the primary database and add a RAM-resident Memcached layer to every node. Check the cache first. Return a hit directly; on a miss, read SQLite, populate the cache, and return the reading. Explain both read and write behavior. Master and slave output must include the latest reading and response time. Supply Makefiles and Bash scripts for applications and supporting services. The suggested directories contain separate database initialization, Memcached initialization, and config.example files for each node type, plus scripts/build_and_run.sh and benchmark_read.sh. Write a benchmark that reads every sensor in two rounds. The first round may miss the cache; the second should hit unless the design causes another outcome. Explain initialization, hit/miss paths, expiration or eviction, any second-round miss, measured round times, and the comparison. The guide must cover Memcached installation/startup, compilation, node execution, requests, benchmark execution, and interpretation.

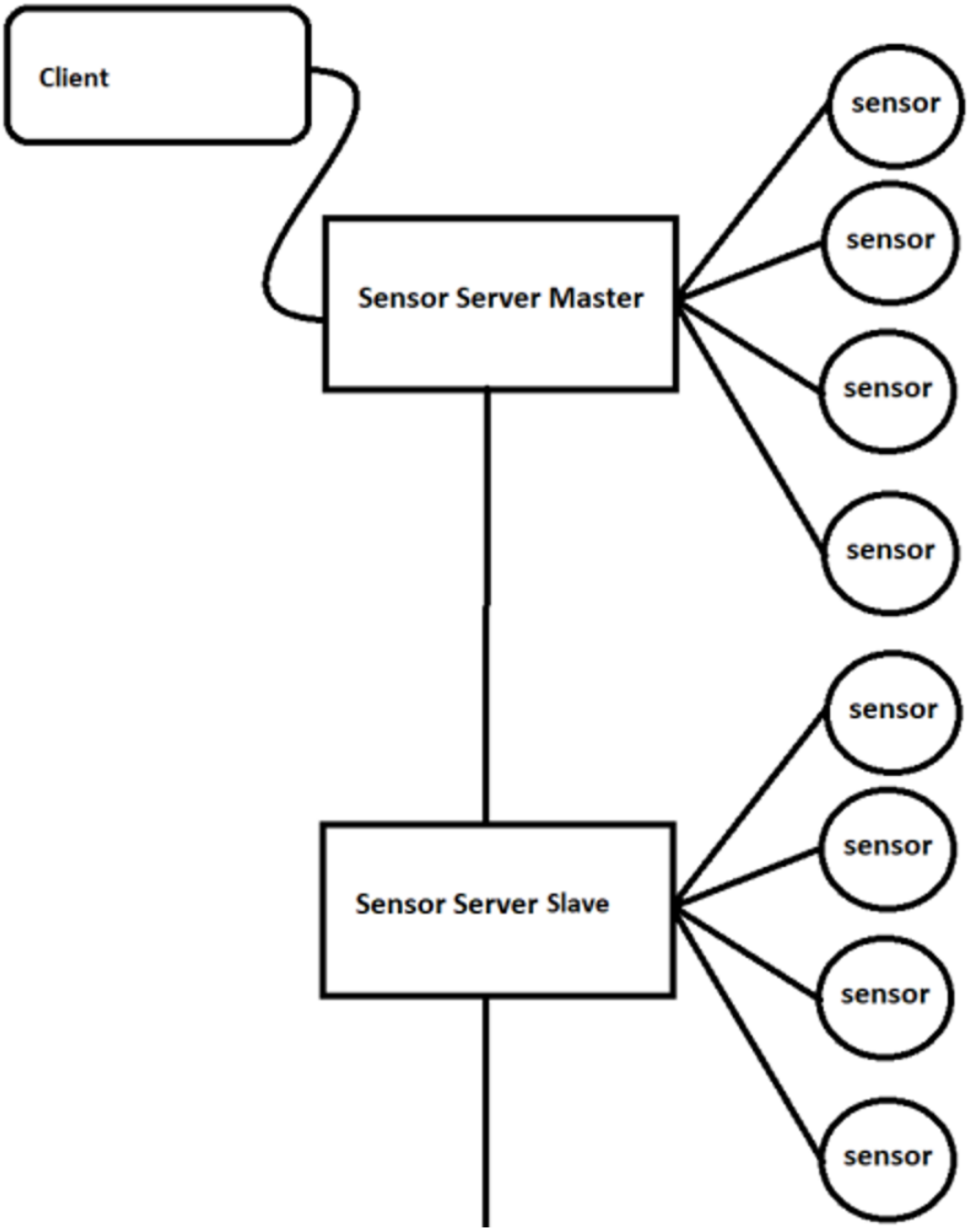


Figure 1: Diagram or recorded result: Part 2: two-layer storage and caching, 20 percent. Panel 1 of 2.

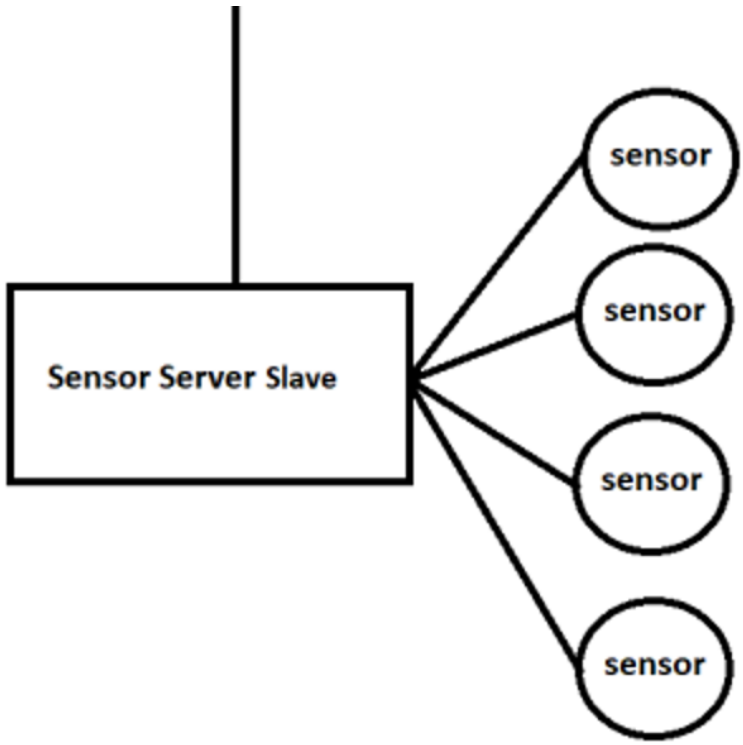


Figure 2: Diagram or recorded result: Part 2: two-layer storage and caching, 20 percent. Panel 2 of 2.

7 Part 3: MQTT gateway, 15 percent

Connect the preceding API to MQTT. Version 3.1.1 or 5 is allowed; identify the chosen version, QoS, and reasons. Run a broker such as Mosquitto. Design and explain request and response topics, publication, subscription, and sensor-read message formats. Write a two-round sensor-call script, measure both round times, and compare them. Submit integration code, Makefile, build/run script, MQTT configuration, test script, installation/configuration guide, and a diagram showing API, broker, master, and slaves. The guide must explain installation, configuration, topics, application startup, mosquitto_pub requests, mosquitto_sub responses, test execution, and timing. Distinguish internal processing time from full client request/response time when analyzing caching.

8 Part 4: SNMP sensor access, 15 percent

Research SNMP and its monitoring role; a short report is required. Use snmpd to expose sensor data. Define custom OIDs. For each sensor, expose at least its name, description, and latest stored value. Use snmpwalk and provide a script that reads all sensors. Configure custom OIDs with snmpd's pass facility. Submit SNMP configuration, custom-OID adapter, all-sensor read script, Makefile where compilation is required, build/run script, and a guide to installation, configuration, and tests. Explain SNMP, OID, MIB, and the acquisition path. The guide must cover package installation, snmpd configuration, OID structure, service startup, snmpwalk use, test script, and the flow from database to SNMP response.

9 Part 5: dated sensor history API, 5 percent

Implement a Mongoose API in C/C++ accepting sensor name, identifier, and a date. Read and return that sensor's values on the specified date from the database in a consistent, readable response. Return a meaningful no-data message when appropriate. Supply Makefile, Bash build/run script, optional database initialization, and Markdown execution/test instructions. The assignment's example requests temperature sensor 101 on 2026-06-01. Its response contains sensor_name, sensor_id, date, and a values array, with examples 10:15:00 -> 24.8 and 10:30:00 -> 25.1. Preserve this as an example contract, not as a hard-coded result. The report must explain request and response formats and the database-reading path.

10 Part 6: persistent alert daemon, 5 percent

Write a background daemon that periodically checks sensor readings and inserts alert events into a database when configured conditions are met. Define the rules and record sensor identifier, sensor name/type, observed value, alert type, time, and status. Include a systemd service, Makefile, Bash build/run script, and installation/start/stop/status guide. Example rules include excessive temperature, humidity outside a permitted range, no data within a timeout, and physically invalid readings. The proposed alerts schema contains id INTEGER PRIMARY KEY AUTOINCREMENT, sensor_id TEXT, sensor_name TEXT, alert_type TEXT, sensor_value TEXT, created_at TEXT, and status TEXT. Explain the daemon's diagram, decision logic, and persistent storage. Submit source, Makefile, service, installation/start script, alert-table initializer, guide, and report. The daemon guide must show compilation; service installation; starting, stopping, and status inspection; system logs; database inspection; and exact alert rules.

11 Report and final-delivery checklist

For every part, describe architecture, components, interconnections, request/response flow, storage, errors, dynamic IP/port management, relevant security analysis, cache/MQTT timing, and a diagram. Verify that all six independent directories have Markdown guides, Makefiles, build/run scripts, initialization and test scripts, diagrams, test outputs, and explanations. Programs must accept changed evaluation data and use only C/C++.